



Politechnika  
Wrocławska

Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka Techniczna

# Implementacja i analiza efektywności algorytmu genetycznego dla problemu komiwojażera

Imię i nazwisko: Wiktor Zięba

Numer albumu: 281056

Wrocław, czerwiec 2026

# Spis treści

|          |                                                                                       |           |
|----------|---------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Wstęp teoretyczny</b>                                                              | <b>2</b>  |
| <b>2</b> | <b>Opis testowanego algorytmu</b>                                                     | <b>2</b>  |
| 2.1      | Idea algorytmu genetycznego . . . . .                                                 | 2         |
| 2.2      | Selekcja . . . . .                                                                    | 3         |
| 2.3      | Krzyżowanie . . . . .                                                                 | 3         |
| 2.4      | Mutacja . . . . .                                                                     | 3         |
| 2.5      | Tworzenie kolejnej populacji . . . . .                                                | 4         |
| 2.6      | Pseudokod algorytmu . . . . .                                                         | 4         |
| <b>3</b> | <b>Plan eksperymentu</b>                                                              | <b>5</b>  |
| 3.1      | Cel eksperymentu . . . . .                                                            | 5         |
| 3.2      | Dane testowe . . . . .                                                                | 5         |
| 3.3      | Parametry algorytmu . . . . .                                                         | 5         |
| 3.4      | Sposób pomiaru czasu i błędu . . . . .                                                | 6         |
| <b>4</b> | <b>Wyniki eksperymentów</b>                                                           | <b>6</b>  |
| 4.1      | Wpływ rozmiaru instancji na czas znalezienia najlepszego rozwiązania i błąd . . . . . | 6         |
| 4.2      | Wpływ rozmiaru populacji . . . . .                                                    | 8         |
| 4.3      | Wpływ metody mutacji . . . . .                                                        | 10        |
| 4.4      | Podsumowanie wyników . . . . .                                                        | 12        |
| <b>5</b> | <b>Literatura</b>                                                                     | <b>12</b> |

# 1 Wstęp teoretyczny

Problem komiwojażera to zagadnienie optymalizacyjne polegające na znalezieniu w pełnym grafie ważonym najkrótszej ścieżki poczynając od punktu startowego grafu która przechodzi przez wszystkie inne wierzchołki dokładnie raz i kończy się w punkcie startowym (cykl Hamiltona). W przypadku asymetrycznego problemu komiwojażera, który jest przedmiotem badania koszt przejścia z wierzchołka X do wierzchołka Y może mieć różną wartość od kosztu przejścia z wierzchołka Y do X. Istnieją dwa podstawowe podejścia rozwiązywania problemu komiwojażera:

- **Algorytmy dokładne** – wyznaczają rozwiązanie optymalne, jednak ich złożoność czasowa rośnie bardzo szybko wraz ze wzrostem liczby wierzchołków co sprawia, że dla większych instancji problemu ich zastosowanie staje się niepraktyczne
- **Algorytmy heurystyczne** - algorytmy pozwalające uzyskać wynik w znacznie krótszym czasie kosztem tego, że nie gwarantują znalezienia rozwiązania optymalnego

Problem komiwojażera należy do klasy problemów NP-trudnych, nie jest znany algorytm wielomianowy znajdujący optymalne rozwiązanie dla każdego przypadku. Wraz ze wzrostem ilości miast, liczba możliwych tras rośnie silnie dla metod opartych na pełnym przeglądzie, co czyni obliczenie optymalnej trasy niemożliwym dla dużych zbiorów danych. Z tego powodu dla większych instancji problemu stosuje się algorytmy heurystyczne oraz metaheurystyczne, które nie gwarantują znalezienia rozwiązania optymalnego, ale pozwalają uzyskać dobre rozwiązanie w akceptowalnym czasie. Jednym z takich podejść jest algorytm genetyczny. Algorytm ten operuje na populacji możliwych tras z których wybiera najlepsze rozwiązania które są następnie krzyżowane i mutowane. Dzięki temu w kolejnych pokoleniach możliwe jest stopniowe poprawianie jakości uzyskiwanych rozwiązań.

## 2 Opis testowanego algorytmu

W ramach projektu zaimplementowano algorytm genetyczny przeznaczony do rozwiązywania problemu komiwojażera. Jest to algorytm metaheurystyczny, który nie sprawdza wszystkich możliwych tras, lecz operuje na zbiorze przykładowych rozwiązań problemu nazywanym populacją. Każde rozwiązanie w populacji reprezentuje jedną możliwą trasę przechodzącą przez wszystkie miasta. W kolejnych iteracjach algorytm tworzy nowe pokolenia osobników przy użyciu selekcji, krzyżowania oraz mutacji. Celem tych operacji jest stopniowe poprawianie jakości rozwiązań znajdujących się w populacji.

### 2.1 Idea algorytmu genetycznego

Algorytm genetyczny rozpoczyna działanie od utworzenia populacji początkowej. Populacja składa się z osobników, gdzie każdy osobnik reprezentuje jedną możliwą trasę komiwojażera.

Działanie algorytmu można opisać następująco:

1. **Utworzenie populacji początkowej** – na początku generowana jest grupa losowych osobników. Dla każdego osobnika tworzona jest lista miast, która zostaje losowo przemieszana, a następnie uzupełniona o wierzchołek startowy na początku i na końcu trasy.
2. **Obliczenie funkcji oceny** – dla każdego osobnika obliczany jest koszt całej trasy. Koszt wyznaczany jest jako suma kosztów przejść pomiędzy kolejnymi miastami znajdującymi się w ścieżce.
3. **Selekcja rodziców** – w każdej kolejnej generacji wybierani są rodzice, z których będą tworzone nowe rozwiązania potomne. W implementacji zastosowano selekcję turniejową.
4. **Krzyżowanie** – z wybranych rodziców tworzone jest nowe rozwiązanie. Krzyżowanie wykonywane jest z określonym prawdopodobieństwem, zależnym od ustawionego współczynnika krzyżowania.
5. **Mutacja** – po krzyżowaniu potomek może zostać zmodyfikowany. Mutacja również wykonywana jest z określonym prawdopodobieństwem. Jej celem jest wprowadzenie dodatkowej losowości i utrzymanie różnorodności populacji.

6. **Aktualizacja najlepszego rozwiązania** – po utworzeniu nowego osobnika obliczany jest jego koszt i sprawdzane jest, czy jest on lepszy od najlepszego rozwiązania znalezione do tej pory.
7. **Utworzenie kolejnej populacji** – po wygenerowaniu odpowiedniej liczby potomków nowa populacja zastępuje poprzednią. Proces powtarzany jest do momentu spełnienia kryterium stopu.

## 2.2 Selekcja

Do wyboru rodziców zastosowano selekcję turniejową. Polega na losowym wybraniu kilku osobników z aktualnej populacji, a następnie wybraniu najlepszego spośród nich.

Działanie selekcji turniejowej można opisać następująco:

1. Losowanych jest kilku kandydatów z populacji.
2. Porównywane są ich koszty tras.
3. Do dalszego tworzenia potomka wybierany jest osobnik o najmniejszym koszcie.

Selekcja turniejowa prowadzi do wyboru lepszych osobników, ale jednocześnie zachowuje element losowości, ponieważ kandydaci do turnieju są wybierani losowo. Dzięki czemu słabsze osobniki również mogą czasami zostać wybrane.

## 2.3 Krzyżowanie

W projekcie zastosowano krzyżowanie Order Crossover. Krzyżowanie OX polega na wybraniu dwóch punktów cięcia w pierwszym rodzicu. Fragment ścieżki znajdujący się pomiędzy tymi punktami zostaje przepisany do dziecka. Następnie pozostałe pozycje w dziecku są uzupełniane miastami z drugiego rodzica, z zachowaniem ich kolejności oraz bez powtarzania miast. Przykład działania krzyżowania OX:

$$X = [0, 1, 2, 3, 4, 5, 0]$$

$$Y = [0, 4, 1, 5, 2, 3, 0]$$

Jeżeli z pierwszego rodzica X zostanie wybrany fragment [2, 3, 4] to dziecko początkowo ma postać:

$$[0, -, 2, 3, 4, -, 0]$$

Następnie brakujące miejsca są uzupełniane miastami z drugiego rodzica Y, z pominięciem miast, które już znajdują się w dziecku. W rodzicu Y kolejność miast wynosi [4, 1, 5, 2, 3]. Miasta 4, 2 oraz 3 zostały już przepisane z pierwszego rodzica, dlatego do dziecka dodawane są tylko miasta 1 oraz 5. Ostatecznie potomek ma postać:

$$[0, 1, 2, 3, 4, 5, 0]$$

## 2.4 Mutacja

W projekcie zastosowano dwa typy mutacji SWAP oraz INVERSION. Mutacja wykonywana jest z określonym prawdopodobieństwem i ma na celu wprowadzenie dodatkowej losowości do populacji. Dzięki temu algorytm może sprawdzać nowe rozwiązania oraz zmniejsza się ryzyko zbyt szybkiego zatrzymania się populacji wokół podobnych tras.

- Mutacja SWAP - polega na wylosowaniu dwóch miast w ścieżce i zamianie ich miejscami. Przykład mutacji:

$$X = [0, 1, 2, 3, 4, 5, 0]$$

Po zamianie miejscami miast 5 oraz 3 otrzymujemy ścieżkę:

$$X = [0, 1, 2, 5, 4, 3, 0]$$

- Mutacja INVERSION - polega na wylosowaniu dwóch pozycji w ścieżce, a następnie odwróceniu kolejności miast znajdujących się pomiędzy tymi pozycjami. Przykład mutacji:

$$X = [0, 1, 2, 3, 4, 5, 0]$$

Jeżeli zostanie wylosowany fragment ścieżki [3,4,5] to po jego odwróceniu trasa będzie miała postać:

$$X = [0, 1, 2, 5, 4, 3, 0]$$

## 2.5 Tworzenie kolejnej populacji

Podczas tworzenia kolejnej populacji najlepszy osobnik znaleziony do danego momentu jest bezpośrednio przenoszony do nowej populacji. Pozostałe miejsca w populacji uzupełniane są przez nowych potomków tworzonych przy użyciu selekcji, krzyżowania i mutacji. Takie podejście zabezpiecza algorytm przed utratą najlepszego znalezionego rozwiązania. Dzięki temu jakość najlepszego zapamiętanego rozwiązania nie pogarsza się w kolejnych generacjach.

## 2.6 Pseudokod algorytmu

```

1  GENETIC_ALGORITHM(M, start, parametry)
2
3      populacja = generowaniePopulacjiPoczątkowej(M, start)
4      najlepszyOsobnik = MIN(populacja, koszt)
5      startCzasu = aktualnyCzas()
6
7      DOPOKI aktualnyCzas() - startCzasu < limitCzasu WYKONAJ
8
9          nowaPopulacja = {}
10         nowaPopulacja = nowaPopulacja U {najlepszyOsobnik}
11
12         DOPOKI |nowaPopulacja| < rozmiarPopulacji WYKONAJ
13
14             rodzic1 = selekcjaTurniejowa(populacja)
15             rodzic2 = selekcjaTurniejowa(populacja)
16             JEZELI losowaLiczba(0, 1) < wspolczynnikKrzyzowania:
17                 dziecko = krzyzowanieOX(rodzic1, rodzic2)
18             W PRZECIWNYM RAZIE:
19                 dziecko = rodzic1
20
21             JEZELI losowaLiczba(0, 1) < wspolczynnikMutacji:
22                 JEZELI metodaMutacji = SWAP:
23                     dziecko = Swap(dziecko)
24                 W PRZECIWNYM RAZIE:
25                     dziecko = Inversion(dziecko)
26
27             dziecko.koszt = koszt(dziecko)
28             nowaPopulacja = nowaPopulacja U {dziecko}
29
30             JEZELI dziecko.koszt < koszt(najlepszyOsobnik):
31                 najlepszyOsobnik = dziecko
32
33         populacja = nowaPopulacja
34
35     sredniKosztPopulacji = srednia(koszt(osobnik) dla osobnik w populacja)
36
37     ZWROC najlepszyOsobnik, sredniKosztPopulacji

```

Tekst 1: Pseudokod algorytmu genetycznego

## 3 Plan eksperymentu

### 3.1 Cel eksperymentu

Celem eksperymentu była analiza efektywności algorytmu genetycznego zastosowanego do problemu ATSP. Badanie obejmowało pomiar czasu działania algorytmu oraz jakości uzyskiwanych rozwiązań mierzonej błędem względnym względem wartości optymalnej. W ramach eksperymentu zbadano:

1. zależność czasu uzyskania wyniku w funkcji rozmiaru instancji oraz zależność funkcji błędu w funkcji rozmiaru instancji
2. wpływ rozmiaru populacji na czas działania algorytmu oraz błąd
3. wpływ zastosowanej metody mutacji na czas działania algorytmu oraz błąd

### 3.2 Dane testowe

Do badań wykorzystano instancje asymetrycznego problemu komiwojażera. Użyto 10 różnych plików o zróżnicowanych rozmiarach. Lista plików ATSP użytych w eksperymencie:

Table 1: Instancje użyte w badaniach

| Nazwa pliku | Liczba miast |
|-------------|--------------|
| br17.atsp   | 17           |
| ftv33.atsp  | 34           |
| ftv38.atsp  | 39           |
| ftv44.atsp  | 45           |
| ftv47.atsp  | 48           |
| ft53.atsp   | 53           |
| ftv55.atsp  | 56           |
| ftv64.atsp  | 65           |
| ftv70.atsp  | 71           |
| ftv170.atsp | 171          |

### 3.3 Parametry algorytmu

W eksperymencie zastosowano wspólny zestaw parametrów bazowych algorytmu. Parametry dobrano tak, aby algorytm dla badanych instancji zwracał rozwiązania mieszczące się w wymaganym zakresie błędu względnego.

Table 2: Podstawowe parametry algorytmu genetycznego

| Parametr                 | Wartość    |
|--------------------------|------------|
| Rozmiar populacji        | 300        |
| Współczynnik mutacji     | 0.1        |
| Współczynnik krzyżowania | 0.8        |
| Metoda selekcji          | turniejowa |
| Rozmiar turnieju         | 3          |
| Metoda krzyżowania       | OX         |
| Metoda mutacji bazowa    | SWAP       |
| Limit czasu              | 900 s      |

W części badającej wpływ rozmiaru populacji zmieniano wyłącznie liczbę osobników w populacji. Porównano populacje o rozmiarach 100, 200 oraz 300 osobników. Pozostałe parametry algorytmu pozostawały bez zmian. W części badającej wpływ metody mutacji porównano dwie metody: SWAP

oraz INVERSION. Pozostałe parametry algorytmu również pozostawały takie same jak w konfiguracji bazowej.

### 3.4 Sposób pomiaru czasu i błędu

Czas mierzono z użyciem `std::chrono::high_resolution_clock`. Dla każdego uruchomienia algorytmu ustawiono limit czasu równy 15 minut. Algorytm wykonywał kolejne generacje do momentu przekroczenia tego limitu lub do wcześniejszego znalezienia rozwiązania optymalnego. W wynikach zapisywano czas uzyskania najlepszego rozwiązania w danym uruchomieniu. Jeżeli najlepszy koszt został znaleziony wcześniej, a późniejsze generacje nie poprawiły już wyniku, w tabeli pozostawał czas znalezienia tego najlepszego rozwiązania. Do pomiaru nie wliczano czasu wczytywania danych z pliku ani czasu zapisu wyników. Dla każdej badanej konfiguracji wykonano po dwa uruchomienia algorytmu. W tabelach przedstawiono wartości średnie z wykonanych powtórzeń. Jako miarę jakości rozwiązania przyjęto błąd względny względem wartości optymalnej długości trasy dla danej instancji. Błąd względny obliczano według wzoru:

$$\delta = \frac{Koszt_{alg} - Koszt_{opt}}{Koszt_{opt}} \cdot 100\%$$

gdzie  $Koszt_{alg}$  oznacza koszt trasy wyznaczonej przez algorytm genetyczny, natomiast  $Koszt_{opt}$  oznacza wartość optymalną dla danej instancji.

## 4 Wyniki eksperymentów

### 4.1 Wpływ rozmiaru instancji na czas znalezienia najlepszego rozwiązania i błąd

W badaniach zastosowano populację o rozmiarze 300 osobników, krzyżowanie OX, selekcję turniejową oraz mutację typu SWAP. Dla każdej instancji wykonano dwa powtórzenia, a w tabeli przedstawiono wartości średnie. W wynikach jako czas podano czas znalezienia najlepszego rozwiązania, a nie całkowity czas pracy algorytmu. Każde pojedyncze uruchomienie trwało do ustawionego limitu czasu wynoszącego 15 minut. Program mógł się zakończyć wcześniej jedynie w momencie znalezienia rozwiązania optymalnego, w przeciwnym wypadku wynikiem jest zapamiętany czas znalezienia najlepszego rozwiązania.

Table 3: Wpływ rozmiaru instancji na czas znalezienia najlepszego rozwiązania i błąd

| Plik   | N   | OPT  | Najlepszy koszt | Błąd [%] | Błąd populacji [%] | Czas do najlepszego [s] |
|--------|-----|------|-----------------|----------|--------------------|-------------------------|
| br17   | 17  | 39   | 39              | 0.00     | 39.54              | 14.43                   |
| ftv33  | 34  | 1286 | 1532            | 19.09    | 23.85              | 12.89                   |
| ftv38  | 39  | 1530 | 1722            | 12.55    | 14.67              | 7.62                    |
| ftv44  | 45  | 1613 | 2086            | 29.29    | 31.54              | 125.42                  |
| ftv47  | 48  | 1776 | 2106            | 18.58    | 20.99              | 356.10                  |
| ft53   | 53  | 6905 | 7887            | 14.22    | 15.85              | 448.65                  |
| ftv55  | 56  | 1608 | 1934            | 20.30    | 22.95              | 535.37                  |
| ftv64  | 65  | 1839 | 2342            | 27.35    | 30.00              | 550.56                  |
| ftv70  | 71  | 1950 | 2728            | 39.92    | 41.86              | 622.40                  |
| ftv170 | 171 | 2755 | 5946            | 115.84   | 118.45             | 718.00                  |

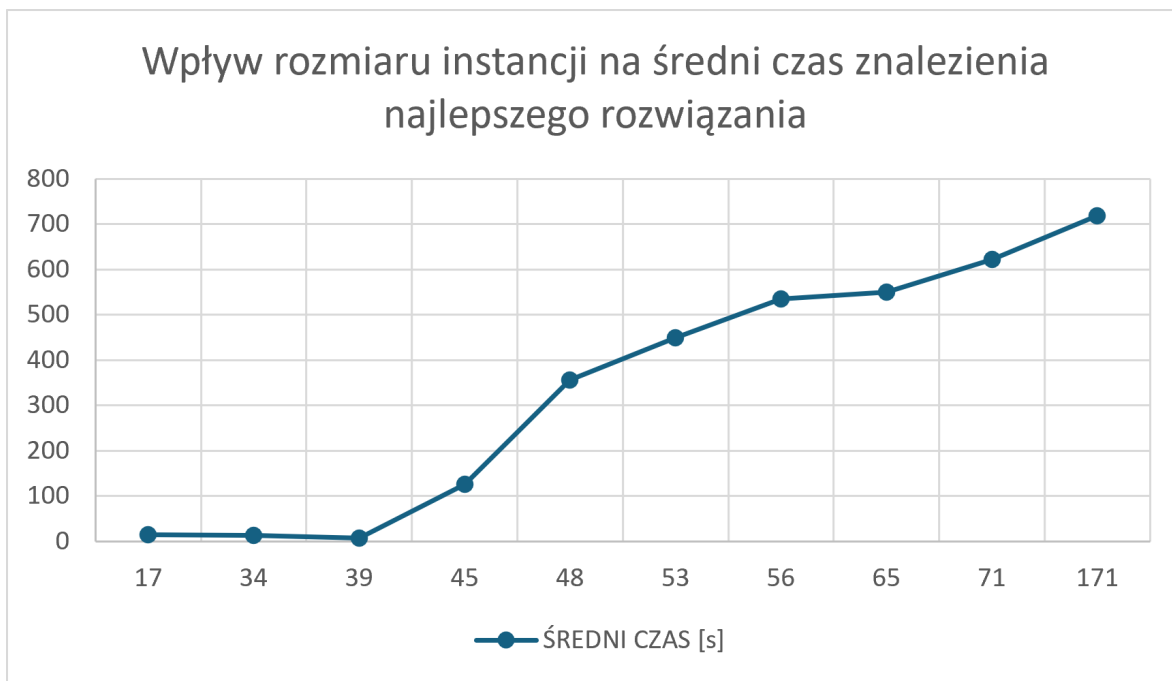


Figure 1: Wpływ rozmiaru instancji na średni czas znalezienia najlepszego rozwiązania

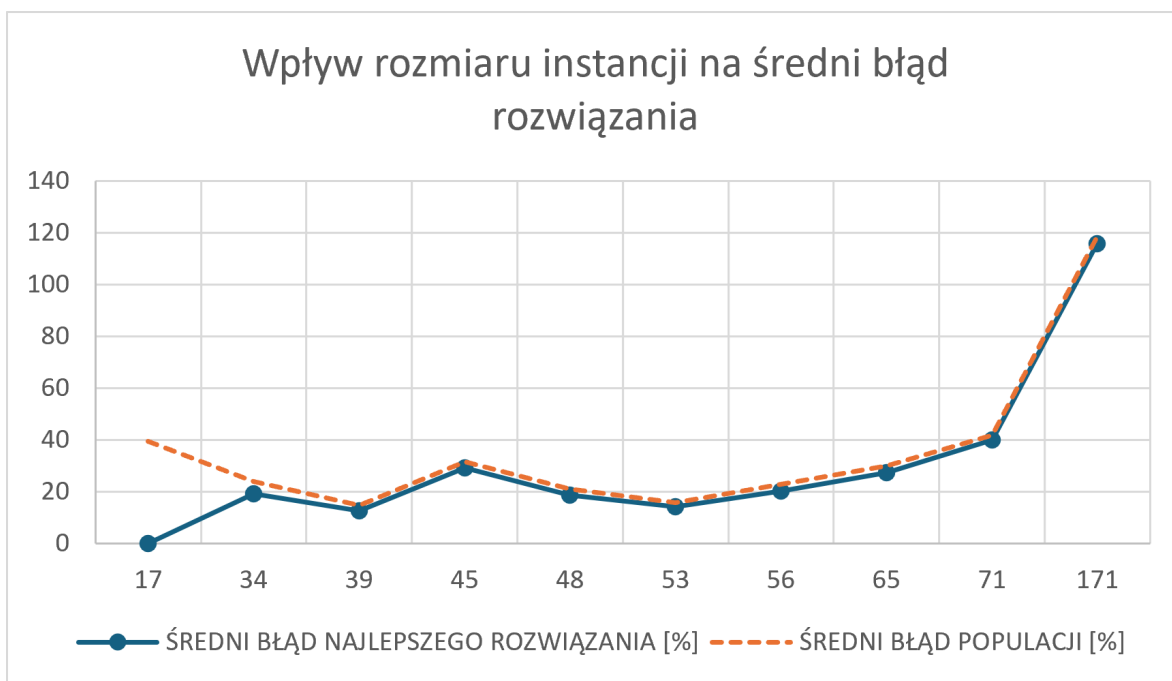


Figure 2: Wpływ rozmiaru instancji na średni błąd względny rozwiązania

Na podstawie uzyskanych danych widoczny jest wzrost czasu potrzebnego do znalezienia najlepszego rozwiązania wraz ze wzrostem rozmiaru instancji. Dla mniejszych instancji najlepsze rozwiązanie było znajdowane stosunkowo szybko, natomiast dla większych plików, takich jak **ftv64**, **ftv70** oraz **ftv170** czas wynosił już kilkaset sekund. Wraz ze wzrostem rozmiaru instancji widoczny jest również wzrost błędu względnego najlepszego znalezionej rozwiązania. Dla najmniejszej instancji **br17** al-



gorytm znalazł rozwiązanie optymalne. Dla instancji średnich błąd zwykle mieścił się w zakresie kilkunastu lub kilkudziesięciu procent. W przypadku instancji `ftv170` błąd najlepszego rozwiązania przekroczył 100%.

## 4.2 Wpływ rozmiaru populacji

W ramach eksperymentu zbadano również wpływ rozmiaru populacji na jakość uzyskiwanego rozwiązania oraz czas znalezienia najlepszego osobnika. Porównano trzy warianty populacji o rozmiarach: 100, 200 i 300 osobników. Pozostałe parametry algorytmu pozostały bez zmian. W tabeli przedstawiono wartości uśrednione dla wszystkich badanych instancji, osobno dla każdej analizowanej wielkości populacji.

Table 4: Wpływ rozmiaru populacji na błąd i czas dla poszczególnych instancji

| Plik                | N   | Błąd [100] [%] | Błąd [200] [%] | Błąd [300] [%] | Czas [100] [s] | Czas [200] [s] | Czas [300] [s] |
|---------------------|-----|----------------|----------------|----------------|----------------|----------------|----------------|
| <code>br17</code>   | 17  | 0.00           | 6.41           | 0.00           | 0.34           | 3.00           | 14.43          |
| <code>ftv33</code>  | 34  | 23.02          | 17.61          | 19.09          | 212.54         | 362.39         | 12.89          |
| <code>ftv38</code>  | 39  | 17.97          | 18.01          | 12.55          | 63.18          | 177.64         | 7.62           |
| <code>ftv44</code>  | 45  | 36.05          | 15.00          | 29.29          | 386.79         | 154.88         | 125.42         |
| <code>ftv47</code>  | 48  | 24.97          | 25.93          | 18.58          | 371.14         | 127.68         | 356.10         |
| <code>ft53</code>   | 53  | 39.59          | 30.54          | 14.22          | 192.32         | 623.55         | 448.65         |
| <code>ftv55</code>  | 56  | 25.16          | 29.14          | 20.30          | 166.76         | 311.59         | 535.37         |
| <code>ftv64</code>  | 65  | 42.31          | 42.28          | 27.35          | 503.90         | 656.12         | 550.56         |
| <code>ftv70</code>  | 71  | 30.15          | 43.74          | 39.92          | 856.20         | 375.03         | 622.40         |
| <code>ftv170</code> | 171 | 126.23         | 94.66          | 115.84         | 857.35         | 519.78         | 718.00         |

Table 5: Średni wpływ rozmiaru populacji na błąd i czas znalezienia najlepszego rozwiązania

| Rozmiar populacji | Błąd najlepszego [%] | Błąd populacji [%] | Czas do najlepszego [s] |
|-------------------|----------------------|--------------------|-------------------------|
| 100               | 36.54                | 41.66              | 361.05                  |
| 200               | 32.33                | 36.94              | 331.16                  |
| 300               | 29.72                | 35.97              | 339.14                  |

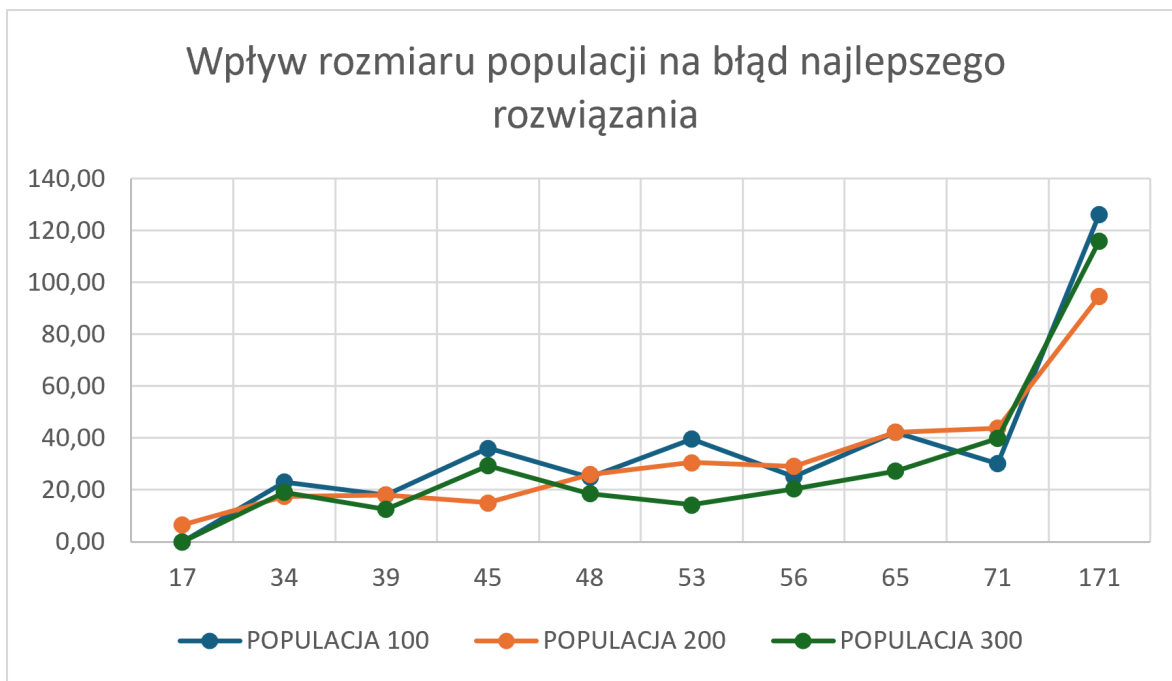


Figure 3: Wpływ rozmiaru populacji na błąd najlepszego rozwiązania

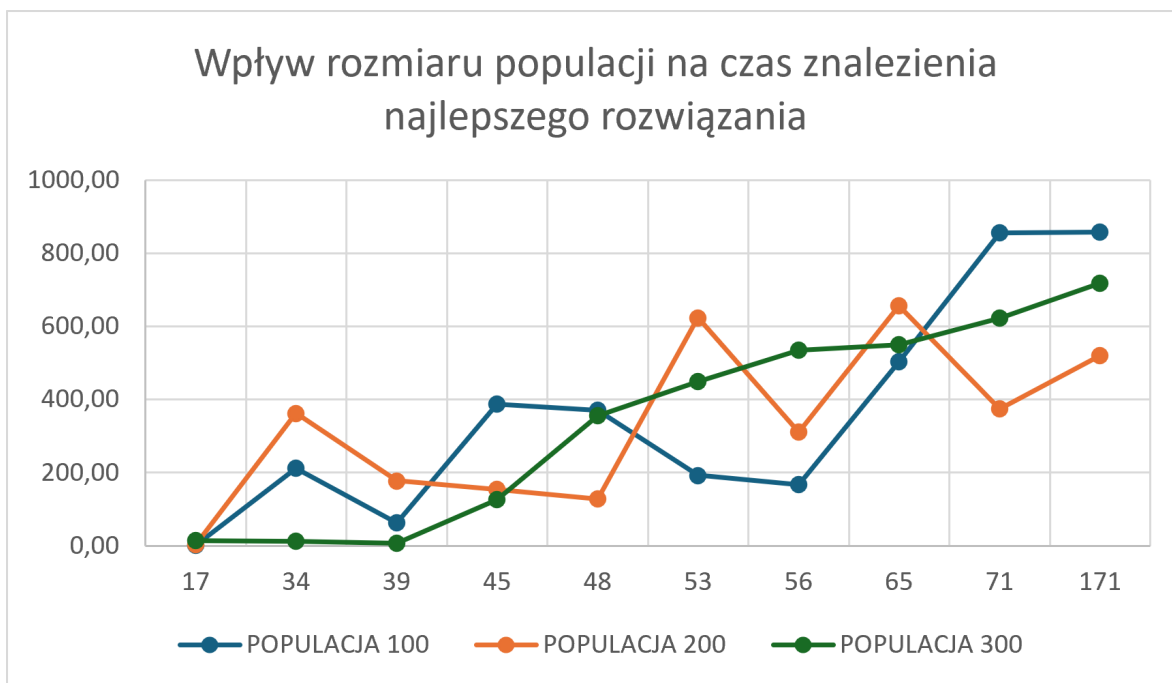


Figure 4: Wpływ rozmiaru populacji na czas znalezienia najlepszego rozwiązania

Zwiększenie rozmiaru populacji średnio poprawiało jakość uzyskiwanych rozwiązań. Najwyższy średni błąd najlepszego osobnika uzyskano dla populacji o rozmiarze 100, gdzie wyniósł on 36.54%. Dla populacji o rozmiarze 200 błąd spadł do 32.33%, natomiast najlepszy średni wynik uzyskano dla populacji 300, dla której średni błąd wyniósł 29.72%. Dla części plików większa populacja dawała wyraźnie lepszy wynik, natomiast dla niektórych instancji różnice były mniejsze lub wyniki były zaburzone przez

losowy charakter algorytmu. Ogólny trend nadal wskazuje jednak, że większa populacja pomagała utrzymać większą różnorodność osobników i zwiększała szansę znalezienia lepszej trasy. Nie zaobserwowano jednoznacznej zależności pomiędzy rozmiarem populacji a czasem znalezienia najlepszego rozwiązania. Zwiększenie populacji powoduje wzrost kosztu obliczeniowego pojedynczej generacji, ale jednocześnie może poprawiać różnorodność rozwiązań i zwiększać szansę wcześniejszego znalezienia dobrej trasy. W rezultacie większa populacja nie musi oznaczać dłuższego czasu do uzyskania najlepszego wyniku.

### 4.3 Wpływ metody mutacji

W projekcie porównano dwie metody mutacji: SWAP oraz INVERSION. W obu przypadkach zastosowano tę samą konfigurację - populację 300 osobników, krzyżowanie OX oraz selekcję turniejową.

Table 6: Wpływ metody mutacji na błąd i czas dla poszczególnych instancji

| Plik   | N   | Błąd SWAP [%] | Błąd INVERSION [%] | Czas SWAP [s] | Czas INVERSION [s] |
|--------|-----|---------------|--------------------|---------------|--------------------|
| br17   | 17  | 0.00          | 0.00               | 14.43         | 0.07               |
| ftv33  | 34  | 19.09         | 7.70               | 12.89         | 263.84             |
| ftv38  | 39  | 12.55         | 15.95              | 7.62          | 458.71             |
| ftv44  | 45  | 29.29         | 16.37              | 125.42        | 545.38             |
| ftv47  | 48  | 18.58         | 24.44              | 356.10        | 465.88             |
| ft53   | 53  | 14.22         | 26.83              | 448.65        | 685.74             |
| ftv55  | 56  | 20.30         | 27.30              | 535.37        | 703.57             |
| ftv64  | 65  | 27.35         | 41.65              | 550.56        | 446.07             |
| ftv70  | 71  | 39.92         | 48.56              | 622.40        | 629.00             |
| ftv170 | 171 | 115.84        | 184.37             | 718.00        | 706.85             |

Table 7: Średni wpływ metody mutacji na błąd i czas znalezienia najlepszego rozwiązania

| Metoda mutacji | Błąd najlepszego [%] | Błąd populacji [%] | Czas do najlepszego [s] |
|----------------|----------------------|--------------------|-------------------------|
| SWAP           | 29.72                | 35.97              | 339.14                  |
| INVERSION      | 39.32                | 51.81              | 490.51                  |

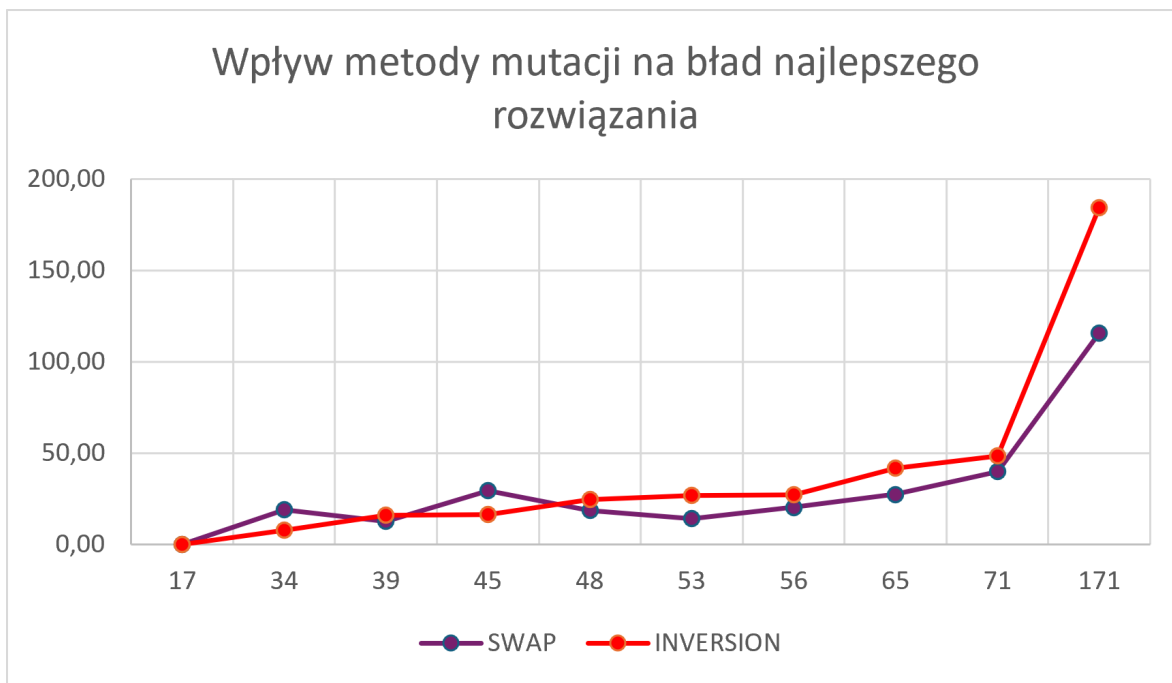


Figure 5: Wpływ metody mutacji na błąd najlepszego rozwiązania

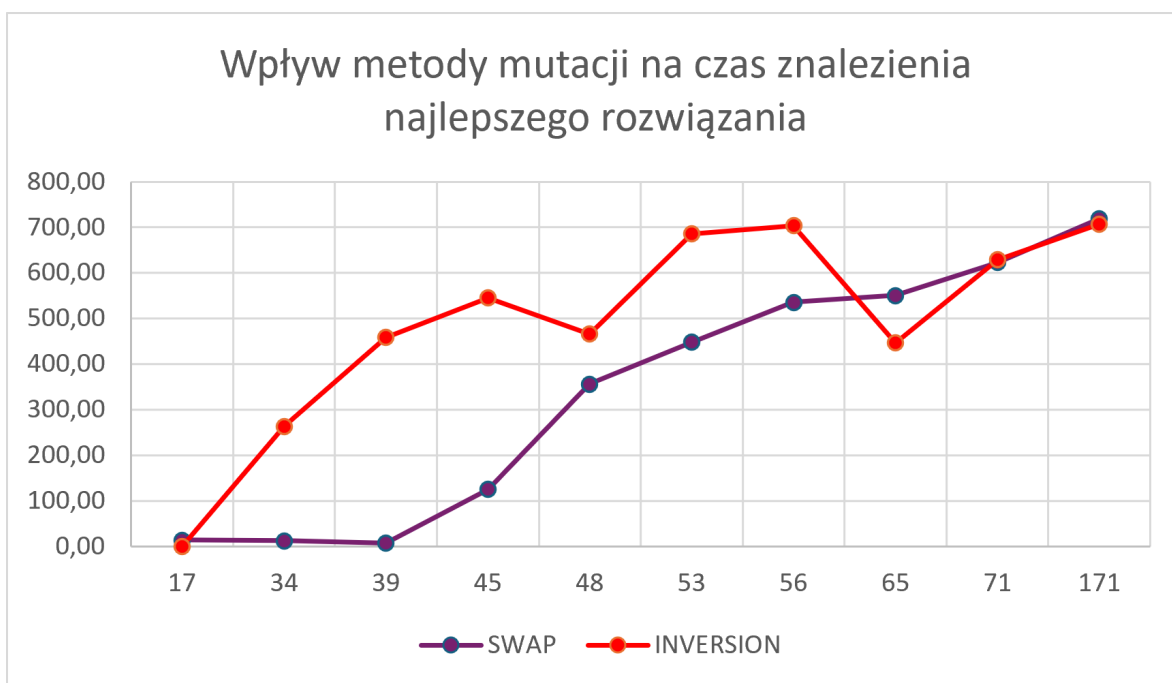


Figure 6: Wpływ metody mutacji na czas znalezienia najlepszego rozwiązania

W przeprowadzonych badaniach lepsze wyniki uzyskano dla mutacji typu SWAP. Średni błąd najlepszego osobnika dla tej metody wyniósł 29.72%, natomiast dla mutacji INVERSION wyniósł 39.32%. Na wykresie można zauważyć, że dla większości instancji mutacja SWAP dawała niższy błąd niż INVERSION. Szczególnie duża różnica pojawiła się dla największej instancji *ftv170*. Mutacja INVERSION wprowadza większą zmianę w strukturze trasy przez odwrócenie fragmentu ścieżki. Mutacja

SWAP wprowadza mniejsze, lokalne zmiany w trasie, ponieważ polega jedynie na zamianie miejscami dwóch miast. W badanych instancjach takie zmiany okazały się korzystniejsze dla zastosowanej konfiguracji algorytmu. Średni czas znalezienia najlepszego rozwiązania był również niższy dla mutacji SWAP. Dla tej metody wyniósł około 339s, natomiast dla mutacji INVERSION 490s. W przeprowadzonych eksperymentach mutacja SWAP była korzystniejsza zarówno pod względem jakości rozwiązania, jak i czasu znalezienia najlepszego osobnika.

#### 4.4 Podsumowanie wyników

Algorytm genetyczny pozwala znaleźć rozwiązania dobrej jakości dla małych i średnich instancji problemu ATSP. Dla najmniejszej instancji **br17** algorytm znalazł rozwiązanie optymalne. Dla większych instancji błąd względny stopniowo wzrastał, a największe trudności wystąpiły dla instancji **ftv170**, gdzie średni błąd przekroczył 100%. Porównanie rozmiarów populacji pokazało, że pod względem jakości rozwiązań najkorzystniejsza była populacja 300 osobników. Uzyskała ona najniższy średni błąd, co można tłumaczyć większą różnorodnością osobników i szerszym przeszukiwaniem przestrzeni możliwych tras. Mniejsze populacje częściej prowadziły do słabszych wyników. W przypadku czasu nie uzyskano jednoznacznej zależności, ponieważ przebieg algorytmu genetycznego jest losowy, a najlepsze rozwiązanie może zostać znalezione zarówno we wcześniejszych, jak i późniejszych etapach działania algorytmu. Porównanie metod mutacji wykazało, że w przeprowadzonych testach korzystniejsza była mutacja SWAP. Dawała ona niższy średni błąd najlepszego rozwiązania, niższy średni błąd ostatniej populacji oraz krótszy średni czas znalezienia najlepszego rozwiązania niż mutacja INVERSION. Oznacza to, że dla zastosowanej konfiguracji algorytmu mniejsze, lokalne zmiany w trasie były skuteczniejsze niż odwracanie większych fragmentów ścieżki. Wniki algorytmu genetycznego są wysoce zależne od losowości oraz dobranych parametrów. Z tego powodu dla tej samej instancji różne uruchomienia mogą dawać różne wyniki. W celu dokładniejszej analizy należałoby wykonać większą liczbę powtórzeń, jednak ze względu na bardzo długi czas obliczeń w projekcie wykonano po dwa powtórzenia dla każdej konfiguracji.

### 5 Literatura

1. Travelling salesman problem, Wikipedia: [https://en.wikipedia.org/wiki/Travelling\\_salesman\\_problem](https://en.wikipedia.org/wiki/Travelling_salesman_problem), dostęp: 10.06.2026.
2. Genetic algorithm, Wikipedia: [https://en.wikipedia.org/wiki/Genetic\\_algorithm](https://en.wikipedia.org/wiki/Genetic_algorithm), dostęp: 10.06.2026.
3. Genetic algorithm, GeeksForGeeks: <https://www.geeksforgeeks.org/dsa/traveling-salesman-problem-using-genetic-algorithm/>, dostęp: 10.06.2026.
4. Algorytmy genetyczne dla problemu komiwojażera: <https://www.aragorn.wi.pb.edu.pl/~wkwedlo/EA5.pdf>, dostęp: 10.06.2026.